



Session 01:

React Fundamentals, Components, Props, State



1. Biết được tổng quan về ReactJS
2. Biết được các tính năng của ReactJS
3. Hiểu được cú pháp React Element, JSX
4. Biết, hiểu được về React Component
5. Sử dụng được React Props / State
6. Hiểu được về Component Life-Cycle

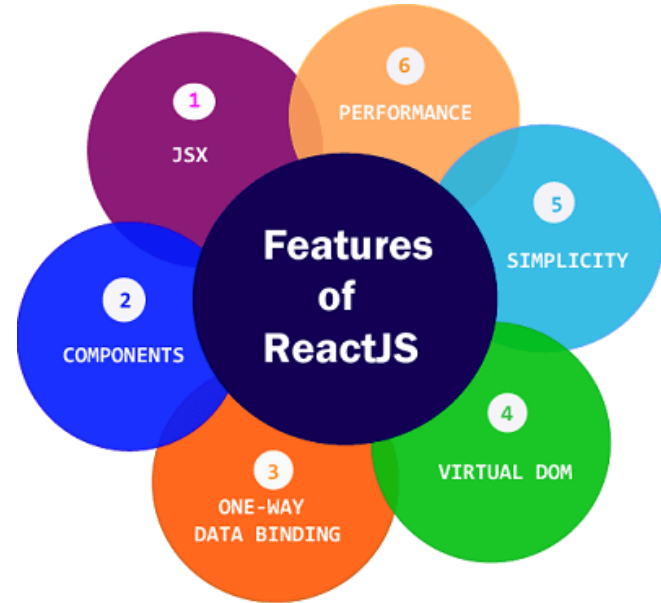
1. Tổng quan về ReactJS

- Là một **thư viện JavaScript** dùng để **xây dựng các thành phần UI** có thể tái sử dụng.
- Được **phát triển bởi Facebook** sử dụng để phát triển ứng dụng **single page, di động,...**
- **Tải, cài đặt dự án ReactJS:**
 - NodeJS: <https://nodejs.org>
 - Visual Studio Code: <https://code.visualstudio.com>
 - **Khởi tạo dự án ReactJS với:**
 - **Vite + JavaScript:** `npm create vite@latest [my-react-app] -- --template react`
 - **Vite + TypeScript:** `npm create vite@latest [my-react-app] -- --template react-ts`

2. Tính năng của ReactJS

JSX (JavaScript XML)

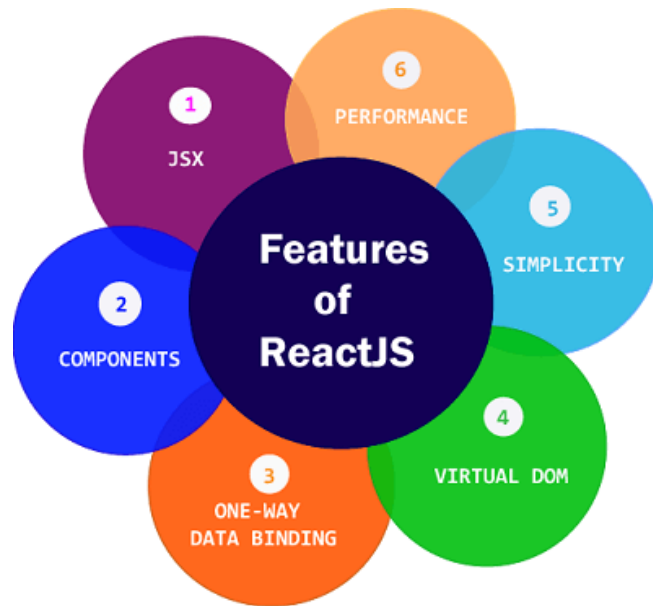
- Cú pháp mở rộng của JavaScript
- Cho phép viết code giống HTML trong file JavaScript
- Công dụng:
 - Dễ đọc, dễ hình dung cấu trúc giao diện
 - Gom **HTML**, **CSS**, **JS** vào 1 nơi gọi là component giúp dễ quản lý hơn
 - Hỗ trợ biểu thức **JavaScript** trong **UI**: nhúng biến, toán tử vào { }
- Lưu ý: các thuộc tính trong **JSX** được viết theo quy tắc **camelCase** (Ví dụ: className, htmlFor,...)



2. Tính năng của ReactJS

Component

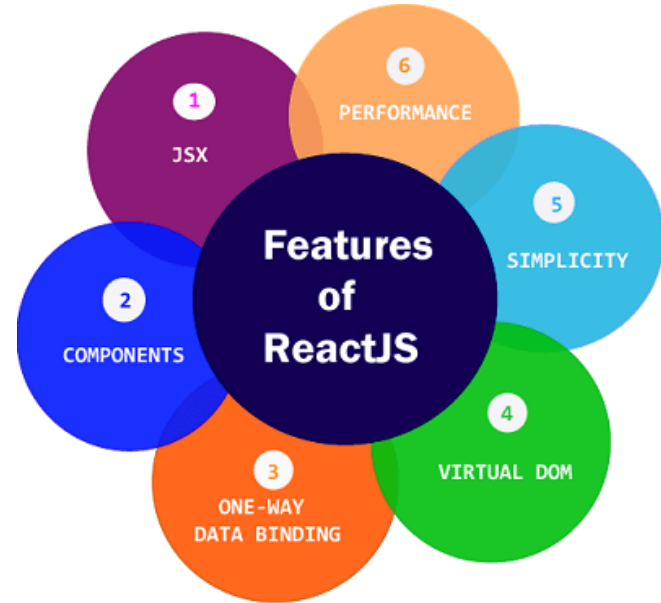
- Là một khối code nhỏ nhất để xây dựng UI trong ReactJS
- Mỗi component tự quản lý logic và giao diện riêng, có thể tái sử dụng ở bất kỳ đâu trong ứng dụng
- Công dụng:
 - Tách giao diện thành các phần nhỏ, dễ quản lý, dễ tái sử dụng
 - Độc lập, tự quản lý logic riêng mỗi component
 - Nhiều component nhỏ có thể ghép lại thành UI lớn



2. Tính năng của ReactJS

One-way Data Binding

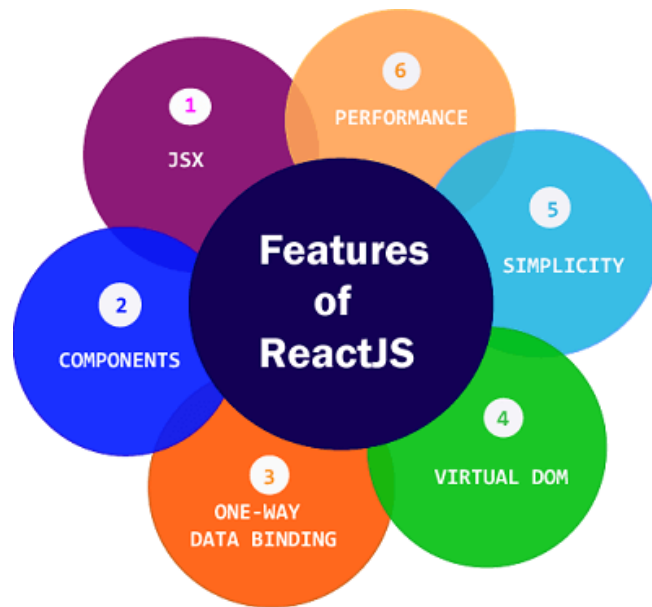
- **Dữ liệu luôn truyền theo một chiều từ component cha sang component con** thông qua **props**
- Component con không được phép thay đổi dữ liệu của component cha trực tiếp
- **Công dụng:**
 - Luồng dữ liệu một chiều giúp kiểm soát hướng đi -> **dễ quản lý dữ liệu**
 - State nằm ở đâu thì UI thay đổi từ đó -> **dễ debug**
 - Dữ liệu một chiều giúp tránh vòng lặp cập nhật vô hạn trong app lớn -> **dễ mở rộng**



2. Tính năng của ReactJS

Virtual DOM

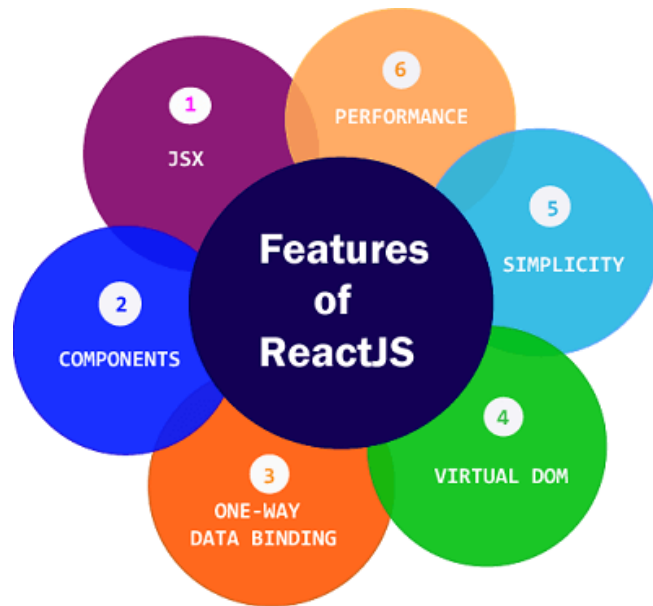
- **Là bản sao** của **Real DOM** được **lưu trong bộ nhớ**
- Giúp React tối ưu hiệu suất, cập nhật giao diện nhanh hơn, ít tốn tài nguyên hơn
- **Công dụng:**
 - Giảm số lần thao tác với DOM thật
 - Chỉ cập nhật chính xác phần UI bị thay đổi
 - Hoạt động mượt hơn, tối ưu hiệu suất



2. Tính năng của ReactJS

Simplicity

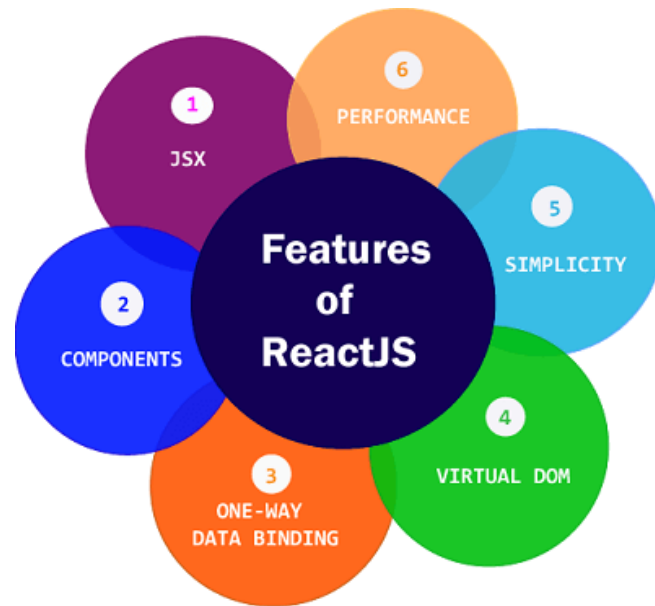
- Là **triết lý cốt lõi của ReactJS**: “Đơn giản hóa cách xây dựng giao diện và quản lý trạng thái”
- React được thiết kế để dễ bắt đầu, dễ đọc, dễ bảo trì, dễ mở rộng, không ép buộc nhiều quy tắc phức tạp
- **Những điểm tạo tính Simplicity của ReactJS**:
 - Component-Based
 - JSX
 - One-Way Data Flow
 - Hooks
 - Virtual DOM



2. Tính năng của ReactJS

Performance

- Là **tập hợp các cơ chế** giúp ứng dụng **ReactJS** chạy **nhANH, mượt và tối ưu việc cập nhật giao diện**
- React được thiết kế để dễ bắt đầu, dễ đọc, dễ bảo trì, dễ mở rộng, không ép buộc nhiều quy tắc phức tạp
- **React tập trung 3 điểm lớn:**
 - Virtual DOM
 - Diffing Algorithm
 - Selective Rendering



3. React Element, JSX

React Element là gì?

- Là đối tượng mô tả **UI** trong **ReactJS**
- Không phải là **React Component**
- Là kết quả của **JSX** hoặc **React.createElement()**



3. React Element, JSX

Đặc điểm React Element

- **Immutable:** Element không thay đổi sau khi đã tạo ra UI
- **Mô tả UI, không phải DOM thật:** chỉ có loại thẻ, props, children
- **Đơn giản:** chỉ là object JS, không có logic, không có state



3. React Element, JSX

Cú pháp React Element

- **Cú pháp:** `React.createElement(type, [props], [...children])`

Ví dụ React Element với cú pháp JSX

- **JSX (React Element):** `const element = <h1>Hello React</h1>;`
- **JSX** được compile thành: `const element = React.createElement("h1", null, "Hello React");`
- **Kết quả:**

```
{  
  type: "h1",  
  props: null,  
  children: "Hello React"  
}
```

3. React Element, JSX

Render React Element

- Sử dụng phương thức **ReactDOM.render()** để render một **React Element** vào **root DOM**
- **Cú pháp:** `ReactDOM.render(element, container[callback])`
- **Ví dụ:**

```
const items = ["JavaScript", "ES6", "TypeScript"];
```

```
const elements = React.createElement("section", { id: "technologies" },
```

```
  React.createElement("h1", null, "Programming Languages"),
```

```
  React.createElement("ul", { className: "technologies" },
```

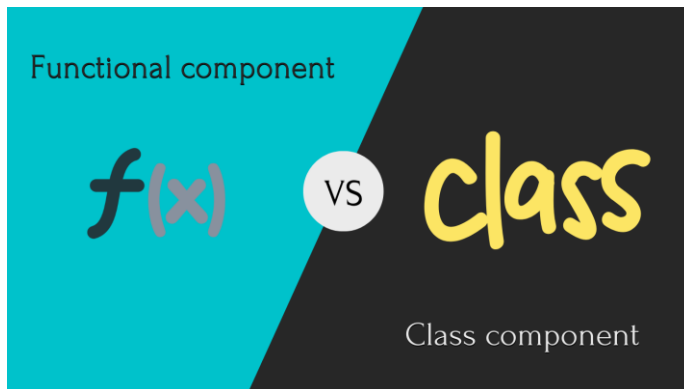
```
    items.map((technology, i) => React.createElement("li", { key: i }, technology));
```

```
  React.render(elements, document.getElementById('root'));
```

4. React Component

Component là gì?

- **Component** là một **block code** độc lập, có thể tái sử dụng
- Khi ứng dụng phình to có hàng ngàn dòng code, **phân tách thành component là việc cần thiết**
- **Mỗi component có code JS, CSS riêng**, có thể tái sử dụng, dễ đọc, dễ viết, dễ test
- Có 2 loại: **Functional Component** (**Stateless**) và **Class Component** (**Stateful**)



4. React Component

Functional Component

- Là một **JavaScript / ES6 function**
- Trả về một **React Element**
- Có thể nhận **props** làm tham số nếu cần
- **Không thể** khởi tạo, cập nhật state (**Stateless**)
- Ví dụ:

```
import React from 'react';

const App: React.FC = () => {
  return <h1>Hello React</h1>;
};

export default App;
```

Class Component

- Là một **JavaScript / ES6 class**
- Kế thừa từ **React Component**
- Có thể nhận **props** trong constructor nếu cần
- **Có thể** khởi tạo, cập nhật state (**Stateful**)
- Ví dụ:

```
import React, { Component } from 'react';

// Định nghĩa kiểu dữ liệu cho Props và State
interface AppProps {}
interface AppState {}

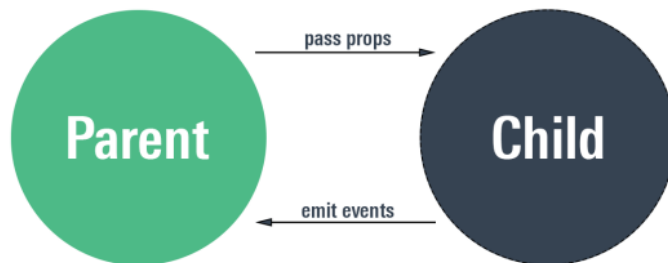
class App extends Component<AppProps, AppState> {
  render() {
    return <h1>Hello React</h1>;
  }
}

export default App;
```

5. React Props / State

Props là gì?

- Là viết tắt của từ **properties** (thuộc tính)
- Là **cách truyền dữ liệu từ component cha sang component con** trong ReactJS
- Được **truyền giống như thuộc tính** (attributes) của **HTML**
- Giá trị của **props** đã được **truyền vào component** thì **không thể thay đổi trực tiếp được**



5. React Props / State

```
import React, { Component } from 'react';

// Định nghĩa kiểu dữ liệu
interface MyComponentProps {}
interface MyComponentState {}

class MyComponent extends Component<MyComponentProps, MyComponentState> {
  render() {
    return (
      <div>
        <h1>Hello</h1>
        <Header id="123" name="Alice" />
      </div>
    );
  }
}

export default MyComponent;
```

```
import React, { Component } from 'react';

// 1. Định nghĩa kiểu dữ liệu cho các props mà Header nhận vào
interface HeaderProps {
  id: string;
  name: string;
}

// 2. Định nghĩa state
interface HeaderState {}

class Header extends Component<HeaderProps, HeaderState> {
  render() {
    return (
      <div>
        <Footer
          id={this.props.id}
          name={this.props.name}
        />
      </div>
    );
  }
}

export default Header;
```

5. React Props / State

Ví dụ Props

```
import React, { Component } from 'react';

// Định nghĩa kiểu dữ liệu cho props dựa trên những gì được truyền từ Header
interface FooterProps {
  id: string;
  name: string;
}

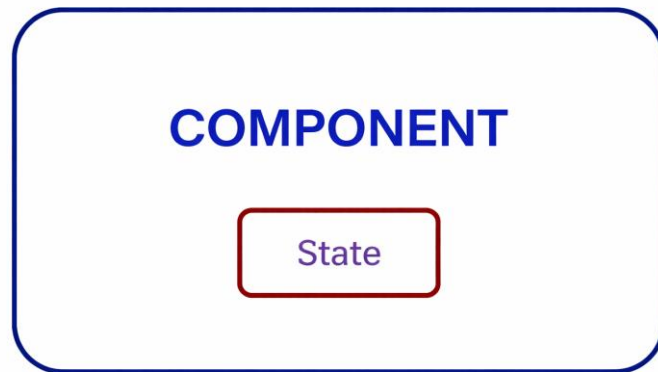
class Footer extends Component<FooterProps> {
  render() {
    return (
      <div>
        <h1>Welcome: {this.props.name}</h1>
        <h1>Id is: {this.props.id}</h1>
      </div>
    );
  }
}

export default Footer;
```

5. React Props / State

State là gì?

- Là một **đối tượng** (object) dùng để **lưu trữ dữ liệu bên trong component**
- **State** bên trong component **có thể bị thay đổi**
- Khi **state thay đổi**, **component** sẽ tự động **render lại giao diện**
- **Mỗi component có state riêng**, độc lập với nhau



5. React Props / State (Ví dụ State)

```
import React, { Component } from 'react';

// 1. Khai báo kiểu dữ liệu
interface MyComponentProps {}
interface MyComponentState {
  id: string;
  name: string;
}

class MyComponent extends Component<MyComponentProps,
MyComponentState> {
  // 2. Khởi tạo State
  constructor(props: MyComponentProps) {
    super(props);
    this.state = { id: '', name: '' };
  }

  // 3. Xử lý Side-effect an toàn, tránh Infinite Loop
  componentDidMount() {
    setTimeout(() => {
      this.setState({ id: '456', name: 'Bob' });
    }, 3000);
  }
}
```

```
// 4. Hàm render trả về giao diện (Pure function)
render() {
  return (
    <div>
      {/* Dữ liệu sẽ tự động cập nhật lại sau 3 giây */}
      <h1>Hello {this.state.name}</h1>
      <h2>Your id is: {this.state.id}</h2>
    </div>
  );
}

export default MyComponent;
```

5. React Props / State

```
import React, { Component } from 'react';

// 1. Định nghĩa Props
interface MyComponentProps {}

class MyComponent extends Component<MyComponentProps> {
  constructor(props: MyComponentProps) {
    super(props);
  }

  render() {
    return (
      <div>
        {/* UI codes */}
        <h1>Đây là giao diện của Component</h1>
      </div>
    );
  }
}

export default MyComponent;
```

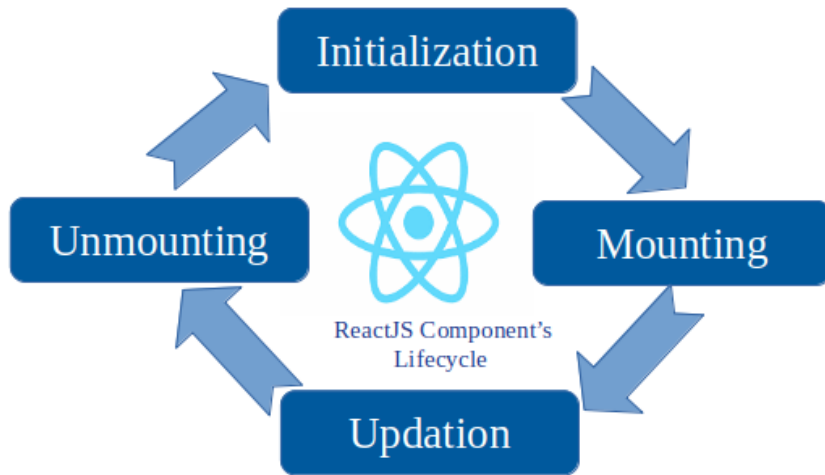
5. React Props / State

So sánh Props và State

Tiêu chí	State	Props
Do component quản lý	Đúng	Đúng
Do component cha quản lý	Không	Đúng
Tự thay đổi trong component	Đúng	Không
Khởi tạo mặc định trong component	Đúng	Đúng
Truyền cho component con	Đúng	Đúng
Làm thay đổi UI khi update	Đúng	Đúng (khi cha truyền props mới)

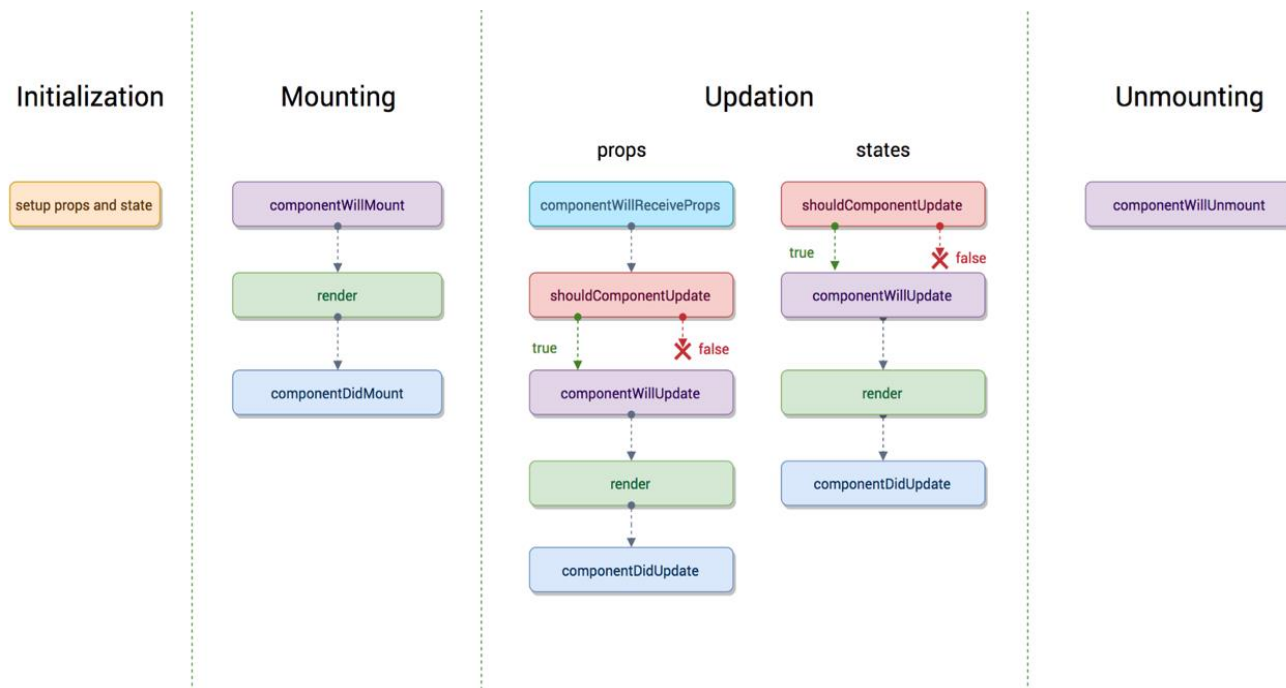
6. Component Life-Cycle

- **Mỗi component** trong React đều có một vòng đời của riêng nó
- **Vòng đời của một Component** có thể được định nghĩa là **một loạt các phương thức được gọi** trong các giai đoạn khác nhau của Component đó



6. Component Life-Cycle

- Mỗi component trong React đều trải qua 4 giai đoạn trong vòng đời như sau:



- ❑ Khái niệm về ReactJS, các tính năng cốt lõi của ReactJS
- ❑ Tải, cài đặt dự án ReactJS đầu tiên với Vite + JavaScript / TypeScript
- ❑ React Element là đối tượng mô tả UI, không phải Component
- ❑ Cú pháp JSX là JavaScript + XML, giúp viết mã HTML trong JavaScript
- ❑ Component là một khối mã độc lập, có thể tái sử dụng
- ❑ Component có 2 loại: Functional Component, Class Component
- ❑ Props là cách thức truyền dữ liệu từ Component cha sang Component con
- ❑ State là một biến cục bộ sử dụng để lưu trữ liệu bên trong Component
- ❑ Vòng đời Component bao gồm 4 giai đoạn: Init, Mounting, Update, Unmount



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN